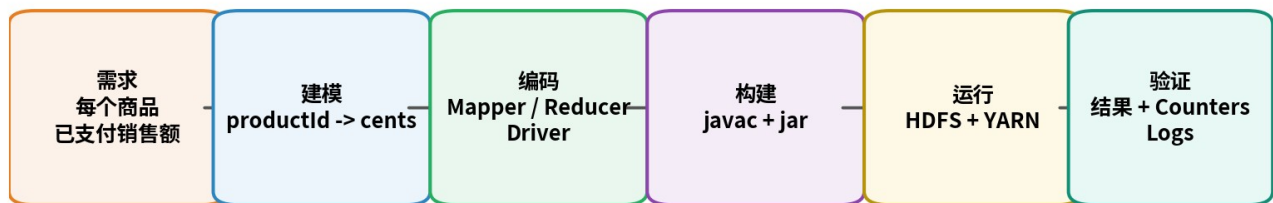


Hadoop MapReduce 原理详解

第十讲：完整实战闭环

销售数据统计 · Java 编程 · HDFS/YARN · Counters · 日志排错

一次完整 MapReduce 实战闭环



目标不是“看懂代码”，而是亲手完成：问题 -> 数据 -> 程序 -> 集群 -> 证据 -> 排错

本讲目标：亲手走通一次“从业务需求到集群结果”的最小 MapReduce 科研/工程闭环。

完整实战：销售数据统计、YARN 运行、Counters 与故障排查

一、本讲要完成什么

- 从一个现实业务需求出发，先独立完成 Key/Value 建模，而不是直接照抄代码。
- 编写一个完整的 Java MapReduce 程序：Driver + Mapper + Reducer + Combiner + Counter。
- 在 Hadoop 3.3.5 / JDK 1.8 环境中完成 javac 编译、Jar 打包、HDFS 上传和 YARN 提交。
- 用“手工小样本结果 + Counters + YARN 日志”三类证据验收 Job，而不是只看到 SUCCESS 就结束。
- 故意制造输入错误、输出目录冲突等故障，建立基础排错顺序。
- 把前九讲中的 InputSplit、Mapper、Shuffle、Combiner、Reducer、YARN、Counter 真正串成一个闭环。

本讲规则：第一次完整闭环建议不要让 AI 替你跳过任何核心步骤：至少亲自创建数据、编译一次、提交一次、看一次结果和日志。以后再交给 AI 自动化。

二、业务需求：统计每个商品“已支付订单”的总销售额

假设电商系统每天产生订单 CSV。现在有一个离线批处理需求：只统计 status=PAID 的订单，计算每个 product_id 的销售总额。

字段	含义	示例
order_id	订单编号	O001
date	日期	2026-09-01
city	城市	Beijing
product_id	商品编号	P1001
quantity	数量	2
amount	本条订单金额（元）	199.80
status	订单状态	PAID / CANCELLED / REFUND

目标输出类似：

期望输出（单位：分）

P1001	46000
P1002	20850
P1003	16000

为什么输出“分”而不是 double 元：金额是十进制定点数。0.1 这类数在二进制浮点中通常不能精确表示。把 199.80 元转成 19980 分，用 long 做加法，逻辑更可靠。

三、先别写代码：用第九讲的方法完成建模

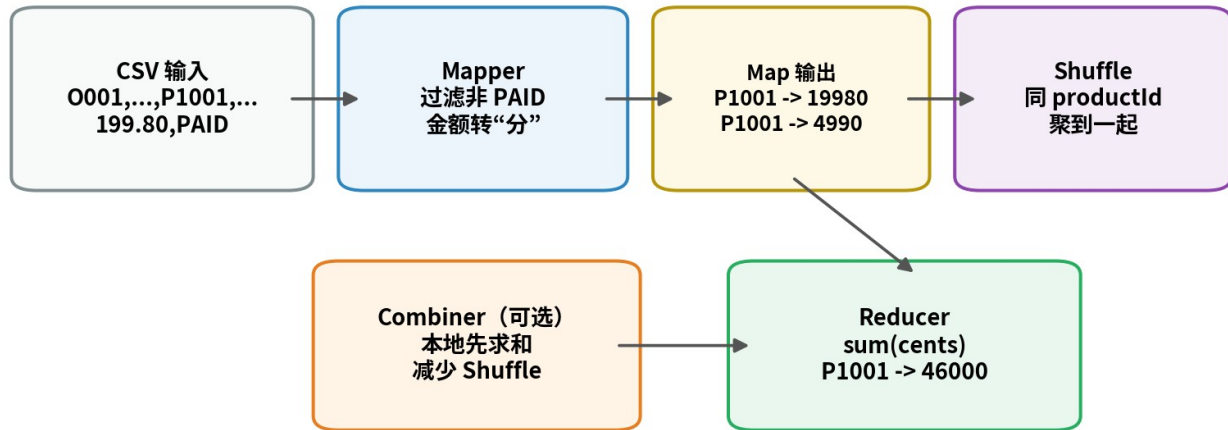
问题	本题答案
最终输出是什么？	<product_id, total_paid_amount_cents>
哪些记录必须见面？	同一个 product_id 的所有 PAID 订单
Map Key 是什么？	product_id
Map Value 是什么？	amount 转换后的 cents
Reduce 做什么？	对同 product_id 的 cents 求和
能否使用 Combiner？	能。整数求和满足结合律/交换律，且输入输出类型一致
是否需要多个 Job？	不需要，一次按商品聚合即可

完整实战：销售数据统计、YARN 运行、Counters 与故障排查

潜在脏数据？

字段不足、金额非法、product_id 为空、非 PAID 状态

销售额统计：数据怎样穿过 MapReduce



货币示例故意不用 double：把 199.80 元转成 19980 分，用整数累加，避免二进制浮点误差。

四、准备一份可手算的小数据

先用 10 条数据，不要一开始就拿大数据。小样本的目的不是测性能，而是验证逻辑。

sales.csv

```

order_id,date,city,product_id,quantity,amount,status
0001,2026-09-01,Beijing,P1001,2,199.80,PAID
0002,2026-09-01,Shanghai,P1002,1,88.50,PAID
0003,2026-09-01,Beijing,P1001,1,49.90,PAID
0004,2026-09-01,Guangzhou,P1003,3,300.00,CANCELLED
0005,2026-09-02,Shanghai,P1002,2,120.00,PAID
0006,2026-09-02,Beijing,P1001,1,10.30,PAID
0007,2026-09-02,Guangzhou,P1003,1,59.90,PAID
0008,2026-09-02,Shanghai,P1002,1,40.00,REFUND
0009,2026-09-03,Guangzhou,P1003,1,100.10,PAID
0010,2026-09-03,Beijing,P1001,2,200.00,PAID
  
```

先手工计算：

商品	被统计的 PAID 金额	总额（元）	总额（分）
P1001	199.80 + 49.90 + 10.30 + 200.00	460.00	46000
P1002	88.50 + 120.00	208.50	20850
P1003	59.90 + 100.10	160.00	16000

验收基准：程序跑出来的结果必须与这张手算表一致；同时 VALID_RECORDS 应为 8，FILTERED_STATUS 应为 2。

完整实战：销售数据统计、YARN 运行、Counters 与故障排查

五、完整 Java 程序：ProductSalesTotal.java

下面这份代码故意加入了 Counter 和基本脏数据防护。你不仅能得到结果，还能知道“究竟跳过了多少条、为什么跳过”。

完整源码

```
import java.io.IOException;
import java.math.BigDecimal;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class ProductSalesTotal {

    // 自定义 Counter: 把“业务层数据质量”也变成可观察指标。
    public enum SalesCounter {
        VALID_RECORDS,
        FILTERED_STATUS,
        BAD_FIELDS,
        BAD_PRODUCT_ID,
        BAD_AMOUNT
    }

    public static class SalesMapper
        extends Mapper<LongWritable, Text, Text, LongWritable> {

        private final Text outKey = new Text();
        private final LongWritable outValue = new LongWritable();

        @Override
        protected void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {

            String line = value.toString().trim();

            // 只在文件第一行命中表头; 避免把 amount 字符串当数字解析。
            if (key.get() == 0 && line.startsWith("order_id,") ) {
                return;
            }

            // -1 表示保留空字段, 例如 a,b,,d 不会把中间空字段吞掉。
            String[] fields = line.split(",", -1);
            if (fields.length != 7) {
                context.getCounter(SalesCounter.BAD_FIELDS).increment(1);
                return;
            }

            String productId = fields[3].trim();
            String amountText = fields[5].trim();
            String status = fields[6].trim();
        }
    }
}
```

```

// 本题只统计已支付订单。
if (!"PAID".equals(status)) {
    context.getCounter(SalesCounter.FILTERED_STATUS).increment(1);
    return;
}

if (productId.isEmpty()) {
    context.getCounter(SalesCounter.BAD_PRODUCT_ID).increment(1);
    return;
}

try {
    // 金额元 -> 分。BigDecimal 避免 double 的二进制浮点误差。
    long cents = new BigDecimal(amountText)
        .movePointRight(2)
        .longValueExact();

    outKey.set(productId);
    outValue.set(cents);
    context.write(outKey, outValue);
    context.getCounter(SalesCounter.VALID_RECORDS).increment(1);

} catch (ArithmeticException | NumberFormatException e) {
    context.getCounter(SalesCounter.BAD_AMOUNT).increment(1);
}
}

public static class SalesReducer
    extends Reducer<Text, LongWritable, Text, LongWritable> {

    private final LongWritable result = new LongWritable();

    @Override
    protected void reduce(Text key, Iterable<LongWritable> values,
        Context context)
        throws IOException, InterruptedException {

        long sum = 0L;
        for (LongWritable value : values) {
            sum += value.get();
        }

        result.set(sum);
        context.write(key, result);
    }
}

public static void main(String[] args) throws Exception {
    if (args.length != 2) {
        System.err.println("Usage: ProductSalesTotal <input> <output>");
        System.exit(2);
    }

    Configuration conf = new Configuration();
    Job job = Job.getInstance(conf, "product paid sales total");

    job.setJarByClass(ProductSalesTotal.class);
    job.setMapperClass(SalesMapper.class);

    // 求和可安全局部聚合，因此复用 Reducer 作为 Combiner。
    job.setCombinerClass(SalesReducer.class);
    job.setReducerClass(SalesReducer.class);

```

```

    job.setMapOutputKeyClass(Text.class);
    job.setMapOutputValueClass(LongWritable.class);
    job.setOutputKeyClass(Text.class);
    job.setOutputValueClass(LongWritable.class);

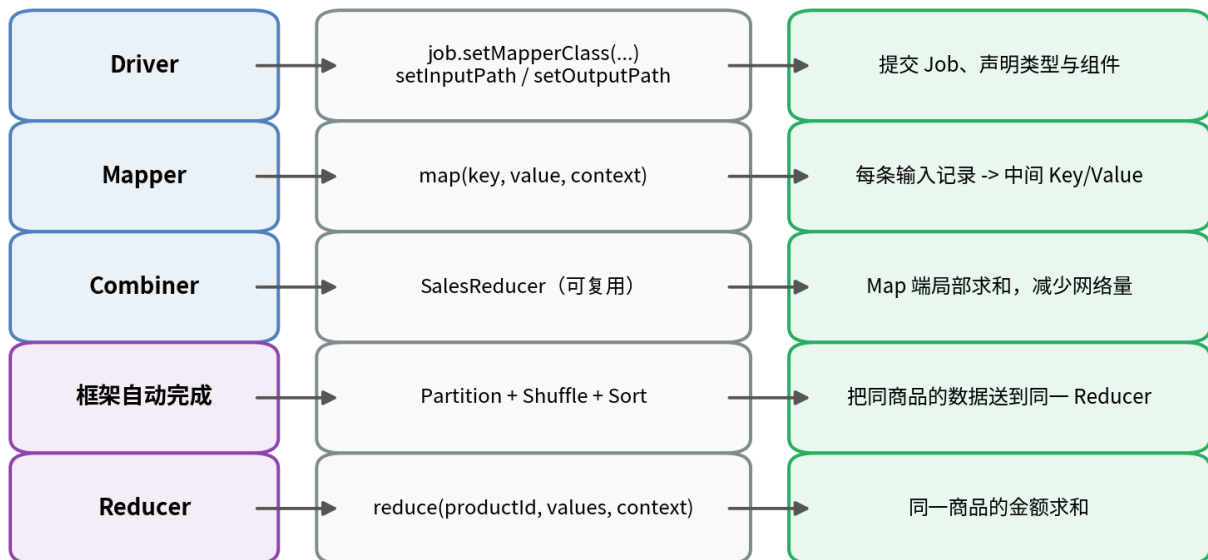
    FileInputFormat.addInputPath(job, new Path(args[0]));
    FileOutputFormat.setOutputPath(job, new Path(args[1]));

    System.exit(job.waitForCompletion(true) ? 0 : 1);
}
}

```

六、把代码重新对应到前面学过的底层机制

你写的 Java 代码，分别对应集群中的哪一段？



代码	底层意义
map(LongWritable, Text, Context)	TextInputFormat 经 RecordReader 生成 “字节偏移量 + 一行文本”
context.write(productId, cents)	写入 Map 端缓冲区，不是直接写 HDFS
setCombinerClass(SalesReducer.class)	Spill/Merge 过程中可进行局部求和，减少中间数据
框架自动 Partition / Shuffle / Sort	相同 productId 的中间记录被送到同一 Reducer
reduce(Text, Iterable<LongWritable>, ...)	一个商品对应一组金额，Reducer 执行求和
FileOutputFormat.setOutputPath(...)	最终结果由 Reducer 写入 HDFS part-r-* 文件

七、上机第 1 步：启动服务并做健康检查

每次重新打开虚拟机都不要假设服务已经运行。先启动，再验证。

完整实战：销售数据统计、YARN 运行、Counters 与故障排查

启动与健康检查

```
# 1) 启动 HDFS
start-dfs.sh

# 2) 启动 YARN
start-yarn.sh

# 3) 检查 Java 进程
jps

# 典型单机/伪分布式环境至少应看到：
# NameNode、DataNode、ResourceManager、NodeManager

# 4) 检查 HDFS 是否可访问
hdfs dfs -ls /

# 5) 检查 YARN 节点
# 如果集群正常，应该能看到可用 NodeManager。
yarn node -list
```

排错原则：如果这一步不正常，不要继续编译/提交 Job。先把基础环境修好，否则后面的错误信息会越来越混乱。

八、上机第 2 步：创建实验目录和 sales.csv

创建测试数据

```
# 在用户主目录创建本讲实验目录
mkdir -p ~/mapreduce-lab10
cd ~/mapreduce-lab10

# 创建测试数据
cat > sales.csv <<'EOF'
order_id,date,city,product_id,quantity,amount,status
0001,2026-09-01,Beijing,P1001,2,199.80,PAID
0002,2026-09-01,Shanghai,P1002,1,88.50,PAID
0003,2026-09-01,Beijing,P1001,1,49.90,PAID
0004,2026-09-01,Guangzhou,P1003,3,300.00,CANCELLED
0005,2026-09-02,Shanghai,P1002,2,120.00,PAID
0006,2026-09-02,Beijing,P1001,1,10.30,PAID
0007,2026-09-02,Guangzhou,P1003,1,59.90,PAID
0008,2026-09-02,Shanghai,P1002,1,40.00,REFUND
0009,2026-09-03,Guangzhou,P1003,1,100.10,PAID
0010,2026-09-03,Beijing,P1001,2,200.00,PAID
EOF

# 先在 Linux 本地确认文件内容
cat sales.csv
```

九、上机第 3 步：编译 Java 并打 Jar

把上一节完整 Java 源码保存为 ProductSalesTotal.java，文件名必须与 public class 名一致。

编译与打包

```
cd ~/mapreduce-lab10

# 用 Hadoop 自己给出的 classpath 编译，避免手工寻找一堆 jar。
mkdir -p classes
javac -classpath "${hadoop classpath}" -d classes ProductSalesTotal.java

# 如果上一步没有报错，classes/ 中会出现 .class 文件。
find classes -type f

# 打包为 jar。-C classes . 表示切换到 classes 目录后把全部 class 放入 jar。
jar -cvf product-sales.jar -C classes .

# 检查 jar 内容
jar -tf product-sales.jar
```

为什么不用 IDE：这一步故意在终端完成，因为它能让你看清“Java 源码 -> .class -> Jar -> Hadoop 提交”的真实链路。以后用 VS Code/IDE 只是把这些步骤自动化。

十、上机第 4 步：把输入上传到 HDFS

HDFS 输入准备

```
# 建立本讲 HDFS 输入目录
hdfs dfs -mkdir -p /user/$USER/mr10/input

# 上传；-f 表示同名文件存在时覆盖
hdfs dfs -put -f sales.csv /user/$USER/mr10/input/

# 验证路径和内容
hdfs dfs -ls /user/$USER/mr10/input
hdfs dfs -cat /user/$USER/mr10/input/sales.csv
```

注意：Linux 本地的 `~/mapreduce-lab10/sales.csv` 和 HDFS 的 `/user/\$USER/mr10/input/sales.csv` 是两个不同文件系统中的对象。

十一、上机第 5 步：提交 YARN 运行

提交 Job

```
# MapReduce 要求输出目录在运行前不存在。
hdfs dfs -rm -r -f /user/$USER/mr10/output

# 提交 Job，并把终端输出同时保存到 run.log。
hadoop jar product-sales.jar ProductSalesTotal /user/$USER/mr10/input /user/$USER/mr10/output 2>&1 | tee run.log
```

完整实战：销售数据统计、YARN 运行、Counters 与故障排查

观察终端：通常会看到 Job ID / Application ID、map/reduce 进度以及最终 Counters。不要只看最后一行。

你应该主动找的证据： map 100% / reduce 100%、Job completed successfully、SalesCounter 中 VALID_RECORDS=8 与 FILTERED_STATUS=2，以及没有 BAD_* 计数。

十二、上机第 6 步：查看结果并与手算值核对

查看结果

```
# 查看输出目录
hdfs dfs -ls /user/$USER/mr10/output

# 查看结果
hdfs dfs -cat /user/$USER/mr10/output/part-r-*

# 预期（行顺序不应作为业务正确性的唯一依据）：
# P1001    46000
# P1002    20850
# P1003    16000
```

不要只看“SUCCESS”：用三类证据验收 Job



SUCCESS 只说明框架认为 Job 完成；
“业务结果正确”还需要手算小样本 + Counters + 日志共同验证。

这里建立一个非常重要的工程习惯：

- 结果证据：输出值是否与手算小样本一致？
- 数据证据：Counter 是否说明真正处理了你以为的那些记录？
- 系统证据：Task 是否发生失败、重试、异常或者 Container 问题？

完整实战：销售数据统计、YARN 运行、Counters 与故障排查

十三、Counters：为什么这是“数据质量仪表盘”

MapReduce 自带很多框架 Counter；我们又定义了业务 Counter。Counter 的价值在于：不必打印几十亿条日志，也能知道全局发生了什么。

Counter	本次预期	解释
VALID_RECORDS	8	真正进入 Map 输出的 PAID 合法订单
FILTERED_STATUS	2	CANCELLED 和 REFUND 被业务规则过滤
BAD_FIELDS	0	CSV 字段数异常
BAD_PRODUCT_ID	0	商品 ID 为空
BAD_AMOUNT	0	金额不是合法十进制金额

可以先在保存下来的运行日志中搜索关键字：

从终端日志中查 Counter

```
grep -E "VALID_RECORDS|FILTERED_STATUS|BAD_FIELDS|BAD_PRODUCT_ID|BAD_AMOUNT" run.log
```

底层意义：Counter 是分布式任务中的“全局聚合观测值”。每个 TaskAttempt 在运行时累加，框架最终汇总，使你能低成本观察大规模数据处理行为。

十四、YARN 日志：程序失败时应该看哪里

如果 Job 失败，首先从提交终端找到 application_... 形式的 Application ID。然后：

查看 YARN 日志

```
# 查看应用基本状态
# 把下面的 ID 替换成你终端里真正看到的 ID。
yarn application -status application_XXXXXXXXXX_0001

# 获取该 Application 的聚合日志
yarn logs -applicationId application_XXXXXXXXXX_0001 > yarn-app.log

# 先搜索常见异常关键词
grep -nE "Exception|ERROR|Caused by|FAILED" yarn-app.log | head -50

# 再打开上下文阅读
less yarn-app.log
```

看日志的方法：不要从几千行日志第一行开始读。先找第一个有意义的 Exception / Caused by，再向上向下看几十行上下文。后面的异常常常只是前一个错误引发的连锁反应。

十五、故意制造错误 1：输出目录已经存在

把同一条 hadoop jar 命令再执行一次，但不要删除 output。你通常会看到输出目录已存在相关错误。

实验：FileAlreadyExistsException

```
# 故意再次运行：output 已存在
hadoop jar product-sales.jar ProductSalesTotal /user/$USER/mr10/input /user/$USER/mr10/output

# 修复：删除旧输出后再运行
hdfs dfs -rm -r -f /user/$USER/mr10/output
```

为什么 Hadoop 不自动覆盖：输出目录代表一个 Job 的提交结果。默认拒绝覆盖可以避免误删上一轮数据，也让 Job 的提交语义更安全。

十六、故意制造错误 2：加入一条坏金额，观察 Counter

实验：业务脏数据不一定要让整个 Job 崩溃

```
# 在本地文件末尾加入一个坏金额；status 仍然是 PAID。
echo '0011,2026-09-03,Beijing,P1004,1,abc,PAID' >> sales.csv

# 重新上传
hdfs dfs -put -f sales.csv /user/$USER/mr10/input/

# 删除旧输出并重跑
hdfs dfs -rm -r -f /user/$USER/mr10/output
hadoop jar product-sales.jar ProductSalesTotal /user/$USER/mr10/input /user/$USER/mr10/output 2>&1 | tee run-bad-amount.log

# 观察 BAD_AMOUNT 应变为 1，但 Job 仍可成功完成。
grep -E "BAD_AMOUNT|VALID_RECORDS" run-bad-amount.log
```

这体现了“数据异常”和“程序异常”的区别。某一条业务记录坏了，可以选择计数后跳过；而 Java 代码崩溃、内存不足等则属于 Task 执行失败。

十七、故意制造错误 3：输入路径写错

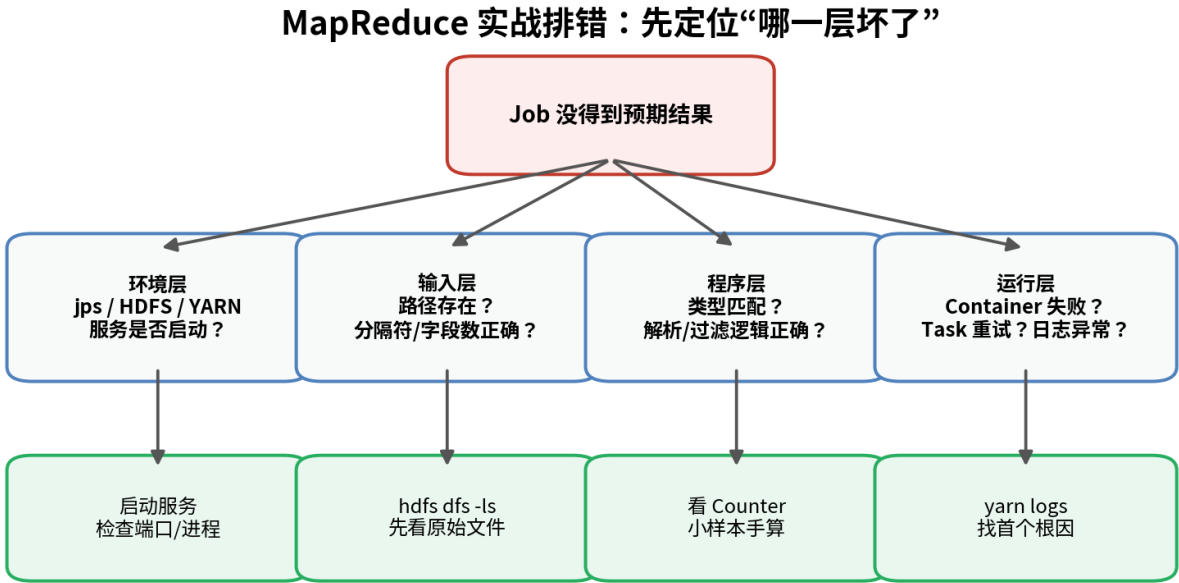
实验：输入路径不存在

```
hadoop jar product-sales.jar ProductSalesTotal /user/$USER/mr10/not-exist /user/$USER/mr10/output2

# 先不要改 Java。直接检查 HDFS：
hdfs dfs -ls /user/$USER/mr10
hdfs dfs -ls /user/$USER/mr10/not-exist
```

要训练的不是“记住报错”：真正要形成的是定位顺序：先问这是环境层、输入层、程序逻辑层，还是运行时层的问题。不同层应该使用不同证据。

十八、一个通用排错框架



原则：不要一看到错误就改代码。先确认“环境、输入、逻辑、运行”哪一层出了问题。

现象	第一步检查	常见根因
命令无法连接 HDFS	jps; hdfs dfs -ls /	NameNode/DataNode 未启动、配置错误
YARN 无节点	jps; yarn node -list	ResourceManager/NodeManager 未启动
输入找不到	hdfs dfs -ls <path>	把 Linux 本地路径误当 HDFS 路径
编译报找不到 Hadoop 类	hadoop classpath; javac 命令	classpath 不完整
Job 提交后 Task 失败	yarn logs	代码异常、资源不足、权限/环境问题
Job SUCCESS 但结果不对	小样本手算 + Counter	Key/Value 建模、过滤条件、解析逻辑错误
Reduce 很慢	YARN/Task 进度、Key 分布	热点 Key、数据倾斜、Shuffle 过大

十九、为什么本题适合使用 Combiner

Mapper 可能产生很多同商品记录。例如一个 Mapper 内部：

局部聚合示意

P1001 -> 19980 P1001 -> 4990 P1001 -> 1030 局部 Combiner 后： P1001 -> 26000
--

只要最终操作是整数求和，局部先求一次和，再跨网络传给 Reducer，不改变结果。

再次强调：不能依赖 Combiner “一定执行”。正确性必须满足：即使 Combiner 完全不运行，Reducer 仍能算出正确答案。

二十、把本次任务扩展成更真实的问题

新需求	需要怎样改建模
每个城市销售额	Key 改为 city, Value 仍为 cents
每天销售额	Key 改为 date
每个城市每天销售额	Key 改为 (city,date) 复合 Key
每个商品销量 (件数)	Value 改为 quantity
每个商品订单数	Value 固定为 1
销售额最高 Top-3 商品	先做本讲聚合, 再增加第二个 Job 做 Top-N
统计坏数据比例	用 Counter 记录 BAD_*, 与总记录数比较
用户表 + 订单表关联	joinKey 改为 user_id, 并引入来源标签

你会发现：Java 框架代码大部分不变，真正变化的是“业务问题 -> Key/Value -> 聚合逻辑”。这正是第九讲建模能力的重要性。

二十一、本讲最小闭环验收清单

- 我能不用看答案，说出为什么 product_id 是 Key、amount cents 是 Value。
- 我亲自手算过 P1001/P1002/P1003 的预期结果。
- 我能解释为什么金额转换成 long cents，而不是直接 double 累加。
- 我能成功用 javac + hadoop classpath 编译，并自己打出 Jar。
- 我能分清 Linux 本地路径与 HDFS 路径。
- 我能成功提交 YARN，并知道 Application ID 去哪里找。
- 我查看过 part-r-*, 并与手算结果逐项核对。
- 我查看过至少一次自定义 Counter。
- 我知道失败时怎样用 yarn logs 找 Exception / Caused by。
- 我亲自制造过至少一个错误，并按照“环境 -> 输入 -> 程序 -> 运行”定位。

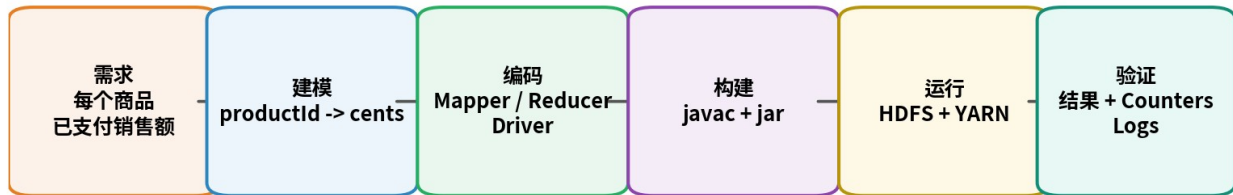
真正学会的标准：不是“这 15 页看懂了”，而是关掉 PDF 后，你仍然能独立把这套小任务从 sales.csv 跑到 HDFS 输出。

二十二、把前十讲串成一张总图

阶段	你现在应该知道什么
问题建模	哪些数据必须见面？Key/Value 如何设计？
输入	HDFS Block、InputSplit、RecordReader 如何把数据交给 Mapper
Map	每条记录如何产生中间 Key/Value
Map 端	缓冲、Spill、Partition、Sort、Combiner
Shuffle	Reducer 如何从多个 Mapper 拉取自己的分区
Reduce	Merge/Group 后如何形成 <key, Iterable<values>>
调度	YARN / ApplicationMaster / Container 如何组织执行
可靠性	TaskAttempt、重试、推测执行、数据倾斜
高级组织	Partitioner、Secondary Sort、TotalOrder、Salting
工程闭环	编译、Jar、HDFS、YARN、Counters、Logs、验收与排错

完整实战：销售数据统计、YARN 运行、Counters 与故障排查

一次完整 MapReduce 实战闭环



目标不是“看懂代码”，而是亲手完成：问题 -> 数据 -> 程序 -> 集群 -> 证据 -> 排错

二十三、本讲常见疑问

Q1：为什么不直接用 Python 写？

Hadoop Streaming 可以用 Python，但本课程用 Java 先看清原生 MapReduce API。掌握原理后再换语言并不难。

Q2：为什么代码看起来比统计逻辑复杂很多？

业务逻辑其实只是过滤 + `<product, cents> + sum`；其余代码是在声明 Hadoop 类型、输入输出和执行组件。

Q3：生产 CSV 能直接 `split(",")` 吗？

严格来说不一定。真实 CSV 可能包含引号、逗号转义等，应使用可靠 CSV 解析库。本讲数据格式受控，故用 `split` 方便理解 MapReduce 主线。

Q4：表头 `key.get() == 0` 总安全吗？

对本讲单个 `TextInputFormat` 文本文件足够用于演示。复杂输入、多个文件或非标准 CSV 应采用更稳健的数据规范/输入处理。

Q5：为什么坏金额不让 Job 直接失败？

这是策略选择。可容忍的脏数据可 `Counter + skip`；关键字段大量异常时则应该设置阈值并让任务失败。

Q6：Counter 会不会很慢？

少量固定 Counter 很常用。不要为每个用户/商品动态创建海量不同 Counter，否则会产生额外开销。

Q7：一个 Reducer 能处理所有商品吗？

可以处理多个 Key，但数据量大时需要多个 Reducer 并行。默认 `Partitioner` 保证同 Key 到同 Reducer。

Q8：`part-r-*` 的顺序为什么可能和我想的不一樣？

完整实战：销售数据统计、YARN 运行、Counters 与故障排查

MapReduce 的业务正确性不应依赖多个输出文件之间的全局排序；全局有序需要额外设计。

Q9: Job 成功是不是就说明结果正确？

不是。SUCCESS 是执行层结论，业务正确性仍要靠小样本、Counter、断言/测试和数据质量检查。

Q10: 以后用 Spark 还需要这些排错思想吗？

需要。输入路径、分区、Shuffle、数据倾斜、脏数据指标、日志和小样本验证仍然存在，只是 API 和执行引擎不同。

二十四、本讲总结

- 现实 MapReduce 任务的第一步是建模，不是写 Java。
- 小样本 + 手工预期值，是验证分布式任务逻辑最便宜、最可靠的第一道防线。
- 本题用 product_id 做 Key，用 long cents 做 Value，用求和 Reducer 完成一次典型批处理聚合。
- Combiner 是性能优化，不是正确性依赖。
- Counter 把业务数据质量变成可观察证据；YARN Logs 负责解释运行失败。
- Job SUCCESS 不等于业务正确，应该用结果、Counter、日志三类证据验收。
- 排错先定位层次：环境 -> 输入 -> 程序逻辑 -> 运行时，不要遇到任何错误都立即改代码。
- 到这里，你已经完成了从 MapReduce 原理到实际集群执行的第一个完整闭环。

下一讲预告：下一讲建议从 MapReduce 过渡到 Spark：先回答“Spark 为什么出现、它到底改掉了 MapReduce 的什么痛点”，再讲 RDD、Transformation/Action、DAG、Stage、Task 与 Shuffle 的对应关系。

参考资料

Apache Hadoop 3.3.x MapReduce Tutorial 与 MapReduce API：用于核对 Job、Mapper、Reducer、Counter、FileInputFormat/FileOutputFormat 等标准语义。

<https://hadoop.apache.org/docs/r3.3.5/hadoop-mapreduce-client/hadoop-mapreduce-client-core/MapReduceTutorial.html>

<https://hadoop.apache.org/docs/stable/api/org/apache/hadoop/mapreduce/package-summary.html>